

2026年度

機械学習演習

第10回（解答編）：演習の解答

計良 宥志

本資料について

第10回の**演習問題の解答**をまとめたもの.

1. **理論演習** : PCA (固有値問題・低ランク近似), 高次元ベクトルの性質 (内積・ノルム)
2. **実装演習** : 課題1 (PCA vs ランダム射影 : 距離保存), 課題2 (同 : 速度比較)

記号 : サンプル数 m , 次元 n . データ行列 $X = (x_1, \dots, x_m)^\top \in \mathbb{R}^{m \times n}$.

A. 理論演習の解答

解答：PCA と固有値問題

問題 $\max_V \text{tr}(V^\top X^\top X V)$ s.t. $V^\top V = I_k$.

ラグランジュ関数 (対称な乗数行列 Λ):

$$L(V, \Lambda) = \text{tr}(V^\top X^\top X V) - \text{tr}(\Lambda(V^\top V - I_k))$$

トレースの微分公式 ($X^\top X$, Λ は対称) より

$$\frac{\partial L}{\partial V} = 2X^\top X V - 2V\Lambda = 0 \implies X^\top X V = V\Lambda$$

- V の各列は $X^\top X$ の固有ベクトル, Λ は固有値の対角行列.
- 目的関数は $\text{tr}(V^\top X^\top X V) = \text{tr}(\Lambda) = \sum$ (選んだ固有値).
- $\Rightarrow$ **最大の k 個**の固有値・固有ベクトルを選べばよい. ■

解答：PCA と低ランク近似

問題 $\min_V \|X - XVV^\top\|_F^2$ s.t. $V^\top V = I_k$.

$P = VV^\top$ は射影行列 ($P^\top = P, P^2 = P$).

$$\|X - XP\|_F^2 = \text{tr}((X - XP)^\top (X - XP)) = \text{tr}(X^\top X) - 2\text{tr}(X^\top XP) + \text{tr}(PX^\top XP)$$

巡回性と $P^2 = P$ より $\text{tr}(PX^\top XP) = \text{tr}(X^\top XP) = \text{tr}(V^\top X^\top XV)$. よって

$$\|X - XVV^\top\|_F^2 = \text{tr}(X^\top X) - \text{tr}(V^\top X^\top XV)$$

- 第 1 項は定数 $\Rightarrow$ **最小化は $\text{tr}(V^\top X^\top XV)$ の最大化** = 前問と同一.
- $\Rightarrow$ 最適 V は $X^\top X$ の上位 k 固有ベクトル. 低ランク近似は PCA で実現される (Eckart–Young). ■

解答：内積の期待値と分散

成分 u_i, v_i は i.i.d., 平均 0, 分散 $1/n$, かつ $u_i \perp v_i$. 各項 $T_i := u_i v_i$ について

$$\mathbb{E}[T_i] = \mathbb{E}[u_i] \mathbb{E}[v_i] = 0, \quad \text{Var}(T_i) = \mathbb{E}[u_i^2] \mathbb{E}[v_i^2] = \frac{1}{n} \cdot \frac{1}{n} = \frac{1}{n^2}$$

独立和 $u^\top v = \sum_{i=1}^n T_i$ なので

$$\mathbb{E}[u^\top v] = \sum_{i=1}^n \mathbb{E}[T_i] = 0, \quad \text{Var}(u^\top v) = \sum_{i=1}^n \text{Var}(T_i) = n \cdot \frac{1}{n^2} = \frac{1}{n}$$

- 平均 0・分散 $\frac{1}{n} \rightarrow 0 \Rightarrow u^\top v \rightarrow 0$ (内積は集中). **ガウス性は不要.** ■

解答：ノルムの期待値と分散

各成分は i.i.d. **ガウス** $N(0, 1/n)$: $\mathbb{E}[u_i] = 0$, $\mathbb{E}[u_i^2] = 1/n$,
 $\mathbb{E}[u_i^4] = 3(\mathbb{E}[u_i^2])^2 = 3/n^2$.

期待値 (分散のみ使用)

$$\mathbb{E}[\|u\|^2] = \sum_{i=1}^n \mathbb{E}[u_i^2] = n \cdot \frac{1}{n} = 1$$

分散 (独立性 + ガウスの 4 次モーメント $\mathbb{E}[u_i^4] = 3/n^2$)

$$\text{Var}(u_i^2) = \mathbb{E}[u_i^4] - (\mathbb{E}[u_i^2])^2 = \frac{3}{n^2} - \frac{1}{n^2} = \frac{2}{n^2}, \quad \text{Var}(\|u\|^2) = \sum_{i=1}^n \text{Var}(u_i^2) = n \cdot \frac{2}{n^2} = \frac{2}{n}$$

- 期待値 $1 \cdot$ 分散 $\frac{2}{n} \rightarrow 0 \Rightarrow \|u\|^2 \rightarrow 1$, すなわち $\|u\| \approx 1$.
- $\mathbb{E}[u_i^4] = 3/n^2$ はガウス特有 (一般は $\text{Var}(\|u\|^2) = (\mathbb{E}[w_i^4] - 1)/n$, $u_i = w_i/\sqrt{n}$). ■

B. 実装演習の解答

解答：課題1 PCA vs ランダム射影（コード）

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from scipy.spatial.distance import pdist

X = load_digits().data                                # (1797, 64)
rng = np.random.default_rng(0)
sub = X[rng.choice(len(X), 200, replace=False)]
d0 = pdist(sub)                                       # 元の距離（全ペア）

for k in [2, 5, 10, 20, 40]:
    Xp = PCA(n_components=k).fit_transform(sub)
    A = rng.normal(0, 1/np.sqrt(k), size=(k, X.shape[1]))
    Xr = sub @ A.T                                   # ランダム射影
    r_pca, r_rp = pdist(Xp)/d0, pdist(Xr)/d0         # 距離比
    # plt.hist(r_pca), plt.hist(r_rp) で k ごとに比較
```

解答：課題1 観察と JL の補題

観点	PCA	ランダム射影
距離比の中心	< 1 に偏る（分散を捨てる）	≈ 1 （不偏）
小さい k	主要構造は保つが距離は縮む	ばらつき大
k を増やすと	単調に改善	分布が 1 に集中（幅 $\sim 1/\sqrt{k}$ ）

- ランダム射影の距離比は平均 1 まわりに集中し，幅は $O(1/\sqrt{k})$ で縮む $\Rightarrow$ **JL の補題**の数値的確認.
- PCA はデータの分散方向を残すため，**低次元構造をもつデータ**では少ない k で有利.
- 「全ペア距離をとにかく保ちたい」「データを見ずに高速に」ならランダム射影.

解答：課題2 高次元での速度比較（コード）

```
import time, numpy as np
from sklearn.decomposition import PCA

rng = np.random.default_rng(0)
m, k = 2000, 50
dims = [1000, 5000, 10000, 50000]
t_pca, t_rp = [], []

for n in dims:
    X = rng.standard_normal((m, n))
    t0 = time.perf_counter()
    PCA(n_components=k).fit_transform(X)          # (a) PCA
    t_pca.append(time.perf_counter() - t0)

    t0 = time.perf_counter()
    A = rng.normal(0, 1/np.sqrt(k), size=(k, n)) # (b) ランダム射影
    _ = X @ A.T
    t_rp.append(time.perf_counter() - t0)
# plt.loglog(dims, t_pca, dims, t_rp) で比較
```

解答：課題2 観察

観点	PCA	ランダム射影
主コスト	共分散/SVD 計算	行列積 $XA^\top$ のみ
次元 n 依存	重い ($O(mn \min(m, n))$ 等)	$O(mnk)$ (ほぼ線形)
データ参照	必要 (学習)	不要

- ランダム射影は n に対してほぼ線形で、次元が大きいほど PCA との速度差が拡大する.
- 速い代わりに距離保存は確率的 (課題2) $\Rightarrow$ **速度と精度の trade-off**.
- 超高次元・大規模データの前処理や近似最近傍などで、ランダム射影は実用的な選択肢.

解答編 おわり